МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
А.С. Кот
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель: освоить возможности языка программирования Python в разработке оконных приложений.

Задание 1:

Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

Должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию. Для всех динамических сцен необходимо задавать параметр скорости!

10) Изобразить в форме приближающийся издали и удаляющийся шар. Шар должен двигаться с постоянной скоростью.

Задание 2:

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

10) Кривая Пеано

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from datetime import datetime
import os
from PIL import Image, ImageGrab
import io

class PeanoCurve:
    """Класс для генерации кривой Пеано"""

    def __init__(self):
        self.code_cache = {}

    def apply_rotation(self, seq, factor):
        return [factor * x for x in seq]

    def conjugate_seq(self, seq):
        return [complex(x.real, -x.imag) for x in seq]

    def generate_code(self, n):
        if n in self.code_cache:
            return self.code_cache[n]

        if n == 0:
            return []

        d0 = [1j, 1j, 1, -1j, -1j, 1, 1j, 1j]

        if n == 1:
            self.code_cache[n] = d0
            return d0

        d_prev = self.generate_code(n-1)
```

```
        d_conj = self.conjugate_seq(d_prev)
        i_d = self.apply_rotation(d_prev, 1j)
        i_conj = self.apply_rotation(d_conj, 1j)

        result = []
        result.extend(d_prev); result.append(1j)
        result.extend(i_d); result.append(1j)
        result.extend(i_conj); result.append(1)
        result.extend(d_conj); result.append(-1j)
        result.extend(self.apply_rotation(d_conj, -1j)); result.append(-1j)
        result.extend(self.apply_rotation(d_prev, -1j)); result.append(1)
        result.extend(d_prev); result.append(1j)
        result.extend(i_d); result.append(1j)
        result.extend(i_conj)

        self.code_cache[n] = result
        return result

    def get_points(self, code, calibre):
        points = [(0, 0)]
        x, y = 0, 0

        for step in code:
            if step == 1:
                x += calibre
            elif step == -1:
                x -= calibre
            elif step == 1j:
                y += calibre
            elif step == -1j:
                y -= calibre
            points.append((x, y))

        return points


class PeanoFractalApp:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Кривая Пеано")
        self.root.geometry("1200x800")

        # Параметры
        self.iterations = tk.IntVar(value=2)
        self.calibre = tk.IntVar(value=10)
        self.speed = tk.IntVar(value=5)
        self.show_squares = tk.BooleanVar(value=False)
        self.is_drawing = False
        self.is_paused = False
        self.current_index = 0
        self.points = []
        self.scaled_points = []

        self.curve = PeanoCurve()

        self.create_widgets()
        self.setup_canvas()

        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

        # Начальный предпросмотр
```

```
        self.root.after(100, self.preview_fractal)

    def create_widgets(self):
        # Панель управления
        control_frame = tk.Frame(self.root, width=280, bg='#f0f0f0')
        control_frame.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)
        control_frame.pack_propagate(False)

        title_label = tk.Label(control_frame, text="Кривая Пеано",
                               font=('Arial', 16, 'bold'), bg='#f0f0f0')
        title_label.pack(pady=10)

        params_frame = tk.LabelFrame(control_frame, text="Параметры",
                                     bg='#f0f0f0', padx=10, pady=10)
        params_frame.pack(fill=tk.X, padx=10, pady=10)

        tk.Label(params_frame,        text="Уровень        рекурсии        (1-4):",
bg='#f0f0f0').pack(anchor=tk.W)
        self.iter_scale = tk.Scale(params_frame, from_=1, to=4,
                                   orient=tk.HORIZONTAL, variable=self.iterations,
                                   command=self.on_param_change)
        self.iter_scale.pack(fill=tk.X)

        tk.Label(params_frame,                text="Размер               элемента:",
bg='#f0f0f0').pack(anchor=tk.W, pady=(10,0))
        self.calibre_scale = tk.Scale(params_frame, from_=5, to=50,
                                      orient=tk.HORIZONTAL, variable=self.calibre,
                                      command=self.on_param_change)
        self.calibre_scale.pack(fill=tk.X)

        tk.Label(params_frame,              text="Скорость              отрисовки:",
bg='#f0f0f0').pack(anchor=tk.W, pady=(10,0))
        self.speed_scale = tk.Scale(params_frame, from_=1, to=10,
                                    orient=tk.HORIZONTAL, variable=self.speed)
        self.speed_scale.pack(fill=tk.X)

        self.squares_check = tk.Checkbutton(params_frame, text="Показывать квадраты
цепи",
                                            variable=self.show_squares,
                                            bg='#f0f0f0',
                                            command=self.on_param_change)
        self.squares_check.pack(anchor=tk.W, pady=10)

        info_frame = tk.LabelFrame(control_frame, text="Информация",
                                   bg='#f0f0f0', padx=10, pady=10)
        info_frame.pack(fill=tk.X, padx=10, pady=10)

        self.info_label = tk.Label(info_frame, text="Не отрисовано",
                                   bg='#f0f0f0', justify=tk.LEFT)
        self.info_label.pack()

        btn_frame = tk.Frame(control_frame, bg='#f0f0f0')
        btn_frame.pack(fill=tk.X, padx=10, pady=10)

        self.draw_btn = tk.Button(btn_frame, text="Начать отрисовку",
                                  command=self.start_drawing,
                                  bg='#4CAF50',   fg='white',   font=('Arial',   10,
'bold'))
        self.draw_btn.pack(fill=tk.X, pady=5)

        self.pause_btn = tk.Button(btn_frame, text="Пауза",
```

```
                                command=self.toggle_pause,
                                bg='#FF9800', fg='white',
                                font=('Arial', 10, 'bold'), state=tk.DISABLED)
        self.pause_btn.pack(fill=tk.X, pady=5)

        self.clear_btn = tk.Button(btn_frame, text="Очистить",
                                command=self.clear_canvas,
                                bg='#f44336', fg='white',
                                font=('Arial', 10, 'bold'))
        self.clear_btn.pack(fill=tk.X, pady=5)

        self.screenshot_btn = tk.Button(btn_frame, text="Скриншот (PNG)",
                                      command=self.take_screenshot,
                                      bg='#2196F3', fg='white',
                                      font=('Arial', 10, 'bold'))
        self.screenshot_btn.pack(fill=tk.X, pady=5)

        progress_frame = tk.Frame(control_frame, bg='#f0f0f0')
        progress_frame.pack(fill=tk.X, padx=10, pady=10)

        tk.Label(progress_frame, text="Прогресс:", bg='#f0f0f0').pack()
        self.progress_bar = ttk.Progressbar(progress_frame, mode='determinate')
        self.progress_bar.pack(fill=tk.X, pady=5)
        self.progress_label = tk.Label(progress_frame, text="0%", bg='#f0f0f0')
        self.progress_label.pack()

    def setup_canvas(self):
        self.canvas_frame = tk.Frame(self.root)
        self.canvas_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        self.canvas = tk.Canvas(self.canvas_frame, bg='white')
        self.canvas.pack(fill=tk.BOTH, expand=True)
        self.canvas.bind('<Configure>', self.on_canvas_resize)

    def on_canvas_resize(self, event=None):
        if not self.is_drawing and self.points:
            self.preview_fractal()

    def on_param_change(self, event=None):
        if not self.is_drawing:
            self.preview_fractal()

    def update_info(self):
        code = self.curve.generate_code(self.iterations.get())
        steps = len(code)

        if self.points:
            xs = [p[0] for p in self.points]
            ys = [p[1] for p in self.points]
            width = max(xs) - min(xs)
            height = max(ys) - min(ys)

            text = f"Уровень: {self.iterations.get()}\n"
            text += f"Шагов: {steps}\n"
            text += f"Размер элемента: {self.calibre.get()}px\n"
            text += f"Скорость: {self.speed.get()}/10\n"
            text += f"Ширина: {width}px\n"
            text += f"Высота: {height}px"
        else:
            text = f"Уровень: {self.iterations.get()}\nШагов: {steps}"
```

```
        self.info_label.config(text=text)

    def get_canvas_bounds(self):
        self.root.update()
        width = self.canvas.winfo_width()
        height = self.canvas.winfo_height()

        if width <= 1:
            width = 900
        if height <= 1:
            height = 700

        margin = 50
        return margin, width - margin, margin, height - margin

    def preview_fractal(self):
        if self.is_drawing:
            return

        code = self.curve.generate_code(self.iterations.get())
        self.points = self.curve.get_points(code, self.calibre.get())

        if not self.points:
            return

        xs = [p[0] for p in self.points]
        ys = [p[1] for p in self.points]
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)

        left, right, top, bottom = self.get_canvas_bounds()
        canvas_width = right - left
        canvas_height = bottom - top

        center_x = (min_x + max_x) / 2
        center_y = (min_y + max_y) / 2

        fractal_width = max_x - min_x
        fractal_height = max_y - min_y

        if fractal_width > 0 and fractal_height > 0:
            scale_x = canvas_width / fractal_width
            scale_y = canvas_height / fractal_height
            scale = min(scale_x, scale_y, 4)
        else:
            scale = 1

        self.scaled_points = []
        for x, y in self.points:
            screen_x = left + canvas_width/2 + (x - center_x) * scale
            screen_y = top + canvas_height/2 - (y - center_y) * scale
            self.scaled_points.append((screen_x, screen_y))

        self.redraw_fractal()
        self.update_info()

    def redraw_fractal(self):
        if not hasattr(self, 'scaled_points') or not self.scaled_points:
            return

        self.canvas.delete("all")
```

```
        if self.show_squares.get():
            size = self.calibre.get() * self.get_current_scale()
            for x, y in self.scaled_points:
                self.canvas.create_rectangle(
                    x - size/2, y - size/2,
                    x + size/2, y + size/2,
                    outline='lightgray', fill=''
                )

        for i in range(len(self.scaled_points) - 1):
            x1, y1 = self.scaled_points[i]
            x2, y2 = self.scaled_points[i + 1]
            self.canvas.create_line(x1, y1, x2, y2, fill='blue', width=2)

    def get_current_scale(self):
        if len(self.points) > 1 and len(self.scaled_points) > 1:
            dx_orig = self.points[1][0] - self.points[0][0]
            dx_scaled = self.scaled_points[1][0] - self.scaled_points[0][0]
            if dx_orig != 0:
                return dx_scaled / dx_orig
        return 1

    def start_drawing(self):
        if self.is_drawing:
            return

        if not hasattr(self, 'scaled_points') or not self.scaled_points:
            self.preview_fractal()

        self.is_drawing = True
        self.is_paused = False
        self.current_index = 0

        total = len(self.scaled_points) - 1
        self.progress_bar['maximum'] = total

        self.canvas.delete("all")

        if self.show_squares.get():
            size = self.calibre.get() * self.get_current_scale()
            for x, y in self.scaled_points:
                self.canvas.create_rectangle(
                    x - size/2, y - size/2,
                    x + size/2, y + size/2,
                    outline='lightgray', fill='', tags='square'
                )

        self.draw_btn.config(state=tk.DISABLED, text="Отрисовка...")
        self.pause_btn.config(state=tk.NORMAL)
        self.clear_btn.config(state=tk.DISABLED)

        self.draw_next()

    def draw_next(self):
        if not self.is_drawing:
            return

        if self.is_paused:
            self.root.after(100, self.draw_next)
            return
```

```
        if self.current_index < len(self.scaled_points) - 1:
            x1, y1 = self.scaled_points[self.current_index]
            x2, y2 = self.scaled_points[self.current_index + 1]

            self.canvas.create_line(x1,   y1,   x2,   y2,   fill='blue',   width=2,
tags='line')
            self.canvas.update()

            self.current_index += 1
            self.progress_bar['value'] = self.current_index

            if len(self.scaled_points) > 1:
                percent = (self.current_index / (len(self.scaled_points) - 1)) * 100
                self.progress_label.config(text=f"{percent:.1f}%")

            delay = max(1, 11 - self.speed.get())
            self.root.after(delay, self.draw_next)
        else:
            self.is_drawing = False
            self.draw_btn.config(state=tk.NORMAL, text="Начать отрисовку")
            self.pause_btn.config(state=tk.DISABLED)
            self.clear_btn.config(state=tk.NORMAL)
            self.progress_label.config(text="Готово!")

    def toggle_pause(self):
        self.is_paused = not self.is_paused
        if self.is_paused:
            self.pause_btn.config(text="Возобновить", bg='#4CAF50')
        else:
            self.pause_btn.config(text="Пауза", bg='#FF9800')

    def clear_canvas(self):
        self.is_drawing = False
        self.is_paused = False
        self.current_index = 0
        self.canvas.delete("all")
        self.progress_bar['value'] = 0
        self.progress_label.config(text="0%")
        self.draw_btn.config(state=tk.NORMAL, text="Начать отрисовку")
        self.pause_btn.config(state=tk.DISABLED)
        self.clear_btn.config(state=tk.NORMAL)

    def take_screenshot(self):
        try:
            if not os.path.exists("screenshots"):
                os.makedirs("screenshots")

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = f"screenshots/peano_{timestamp}.png"

            # Обновляем окно чтобы получить актуальные размеры
            self.root.update()

            # Получаем координаты canvas относительно экрана
            x = self.canvas.winfo_rootx()
            y = self.canvas.winfo_rooty()
            x1 = x + self.canvas.winfo_width()
            y1 = y + self.canvas.winfo_height()

            # Делаем скриншот именно области canvas
```

```
            screenshot = ImageGrab.grab(bbox=(x, y, x1, y1))
            screenshot.save(filename)

            messagebox.showinfo("Скриншот",                                f"Скриншот
сохранён:\n{os.path.abspath(filename)}")
        except Exception as e:
            messagebox.showerror("Ошибка",       f"Не       удалось       сохранить
скриншот:\n{str(e)}")

    def on_closing(self):
        if messagebox.askokcancel("Выход", "Вы уверены, что хотите выйти?"):
         self.root.destroy()

    def run(self):
        self.root.mainloop()


# Задание 1: Шар, приближающийся и удаляющийся
class MovingBallApp:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Движущийся шар")
        self.root.geometry("1200x800")

        # Параметры
        self.speed = tk.DoubleVar(value=3.0)
        self.ball_color = tk.StringVar(value="red")
        self.ball_size_min = tk.IntVar(value=20)
        self.ball_size_max = tk.IntVar(value=120)
        self.is_moving = True
        self.direction = 1
        self.x_pos = 100
        self.animation_id = None

        # Для удаления старых объектов
        self.ball = None
        self.highlight = None

        self.create_widgets()
        self.setup_canvas()

        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

        # Запуск анимации
        self.start_animation()

    def create_widgets(self):
        # Панель управления
        control_frame = tk.Frame(self.root, width=280, bg='#f0f0f0')
        control_frame.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)
        control_frame.pack_propagate(False)

        title_label = tk.Label(control_frame, text="Движущийся шар",
                               font=('Arial', 16, 'bold'), bg='#f0f0f0')
        title_label.pack(pady=10)

        # Рамка параметров
        params_frame = tk.LabelFrame(control_frame, text="Параметры",
                                     bg='#f0f0f0', padx=10, pady=10)
        params_frame.pack(fill=tk.X, padx=10, pady=10)
```

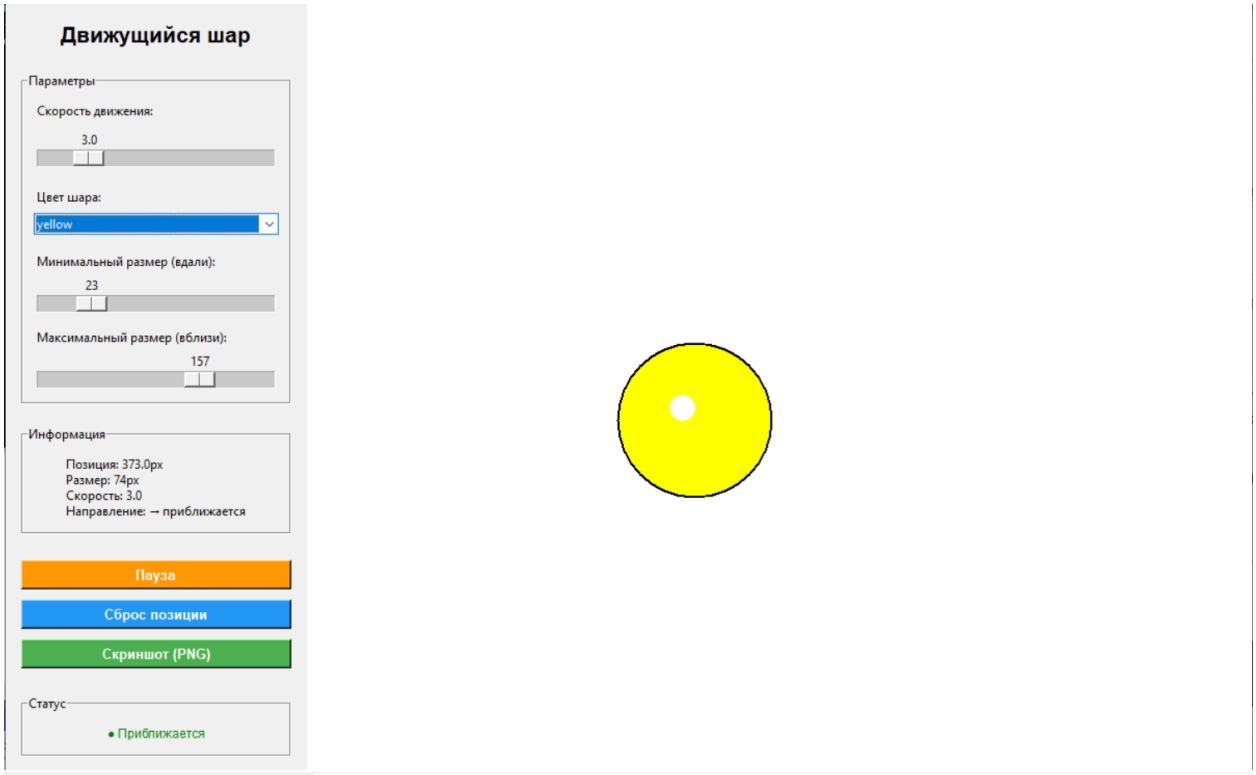
```
        # Скорость
        tk.Label(params_frame,              text="Скорость              движения:",
bg='#f0f0f0').pack(anchor=tk.W)
        self.speed_scale = tk.Scale(params_frame, from_=0.5, to=15.0,
                                    orient=tk.HORIZONTAL, variable=self.speed,
                                    resolution=0.5, length=200)
        self.speed_scale.pack(fill=tk.X, pady=5)

        # Цвет шара
        tk.Label(params_frame,  text="Цвет  шара:",  bg='#f0f0f0').pack(anchor=tk.W,
pady=(10,0))
        self.color_combo = ttk.Combobox(params_frame, textvariable=self.ball_color,
                                        values=['red',  'blue',  'green',  'orange',
'purple',
                                                'pink',  'brown',  'cyan',  'magenta',
'yellow'],
                                        state='readonly', width=15)
        self.color_combo.pack(fill=tk.X, pady=5)
        self.color_combo.bind('<<ComboboxSelected>>', lambda e: self.update_ball())

        # Минимальный размер
        tk.Label(params_frame,       text="Минимальный       размер       (вдали):",
bg='#f0f0f0').pack(anchor=tk.W, pady=(10,0))
        self.min_size_scale = tk.Scale(params_frame, from_=10, to=80,
                                       orient=tk.HORIZONTAL,
variable=self.ball_size_min,
                                       command=lambda x: self.update_ball())
        self.min_size_scale.pack(fill=tk.X)

        # Максимальный размер
        tk.Label(params_frame,      text="Максимальный      размер      (вблизи):",
bg='#f0f0f0').pack(anchor=tk.W, pady=(10,0))
        self.max_size_scale = tk.Scale(params_frame, from_=50, to=200,
                                       orient=tk.HORIZONTAL,
variable=self.ball_size_max,
                                       command=lambda x: self.update_ball())
        self.max_size_scale.pack(fill=tk.X)

        # Информация
        info_frame = tk.LabelFrame(control_frame, text="Информация",
                                   bg='#f0f0f0', padx=10, pady=10)
        info_frame.pack(fill=tk.X, padx=10, pady=10)

        self.info_label = tk.Label(info_frame, text="Ожидание",
                                   bg='#f0f0f0', justify=tk.LEFT)
        self.info_label.pack()

        # Кнопки управления
        btn_frame = tk.Frame(control_frame, bg='#f0f0f0')
        btn_frame.pack(fill=tk.X, padx=10, pady=10)

        self.pause_btn = tk.Button(btn_frame, text="Пауза",
                                   command=self.toggle_motion,
                                   bg='#FF9800', fg='white',
                                   font=('Arial', 10, 'bold'))
        self.pause_btn.pack(fill=tk.X, pady=5)

        self.reset_btn = tk.Button(btn_frame, text="Сброс позиции",
                                   command=self.reset_ball,
                                   bg='#2196F3', fg='white',
                                   font=('Arial', 10, 'bold'))
```

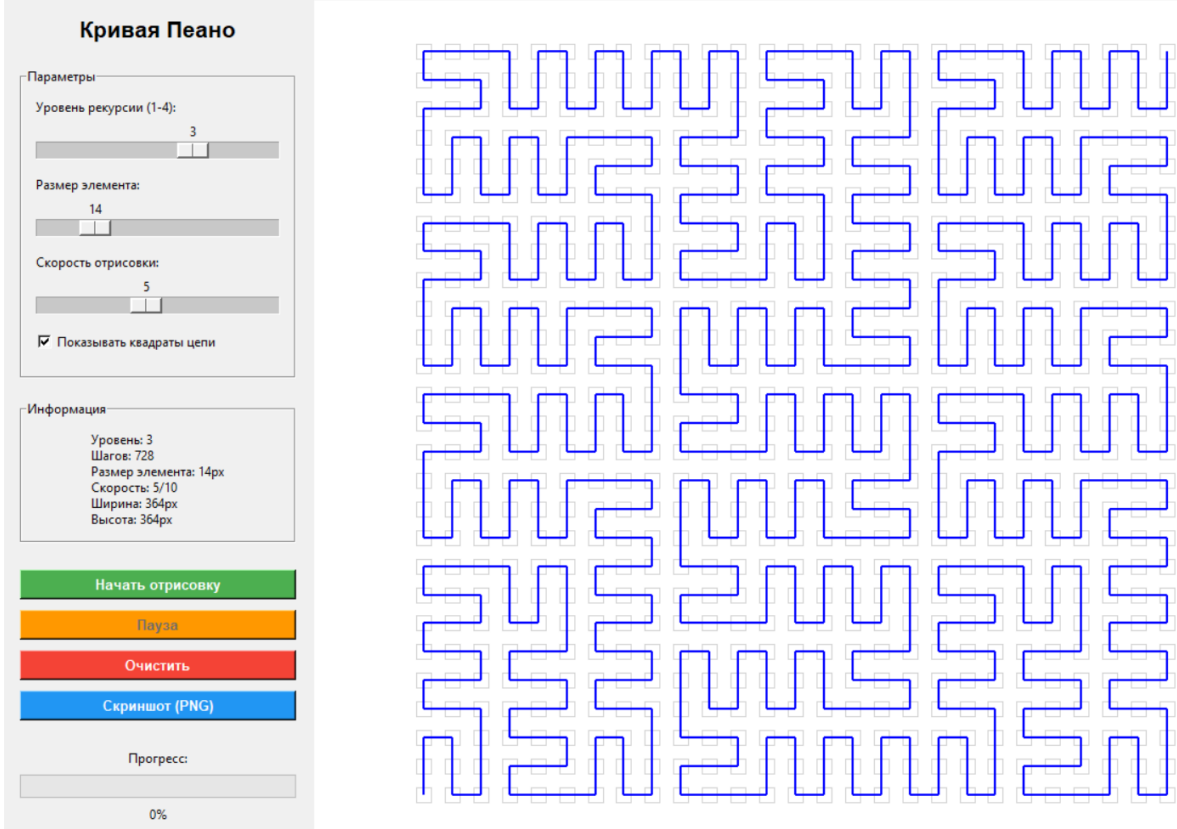
```
        self.reset_btn.pack(fill=tk.X, pady=5)

        self.screenshot_btn = tk.Button(btn_frame, text="Скриншот (PNG)",
                                        command=self.take_screenshot,
                                        bg='#4CAF50', fg='white',
                                        font=('Arial', 10, 'bold'))
        self.screenshot_btn.pack(fill=tk.X, pady=5)

        # Дополнительная информация
        status_frame = tk.LabelFrame(control_frame, text="Статус",
                                     bg='#f0f0f0', padx=10, pady=10)
        status_frame.pack(fill=tk.X, padx=10, pady=10)

        self.status_label = tk.Label(status_frame, text="● Движение активно",
                                     bg='#f0f0f0', fg='green', font=('Arial', 9))
        self.status_label.pack()

    def setup_canvas(self):
        self.canvas_frame = tk.Frame(self.root)
        self.canvas_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        self.canvas = tk.Canvas(self.canvas_frame, bg='white', highlightthickness=0)
        self.canvas.pack(fill=tk.BOTH, expand=True)
        self.canvas.bind('<Configure>', self.on_canvas_resize)

    def on_canvas_resize(self, event=None):
        self.update_ball()

    def get_canvas_width(self):
        self.root.update()
        width = self.canvas.winfo_width()
        if width <= 1:
            width = 900
        return width

    def get_canvas_height(self):
        self.root.update()
        height = self.canvas.winfo_height()
        if height <= 1:
            height = 600
        return height

    def get_ball_size(self):
        canvas_width = self.get_canvas_width()
        min_x = 100
        max_x = canvas_width - 100
        progress = (self.x_pos - min_x) / (max_x - min_x)
        progress = max(0, min(1, progress))
        size    =    self.ball_size_min.get()    +    (self.ball_size_max.get()    -
self.ball_size_min.get()) * progress
        return int(size)

    def update_ball(self):
        # Полностью очищаем canvas
        self.canvas.delete("all")

        canvas_width = self.get_canvas_width()
        canvas_height = self.get_canvas_height()
        center_y = canvas_height // 2
        size = self.get_ball_size()
```

```
        x1 = self.x_pos - size
        y1 = center_y - size
        x2 = self.x_pos + size
        y2 = center_y + size

        # Рисуем шар
        self.ball = self.canvas.create_oval(x1, y1, x2, y2,
                                            fill=self.ball_color.get(),
                                            outline='black', width=2)

        # Блик
        highlight_size = size // 3
        self.highlight = self.canvas.create_oval(
            self.x_pos - highlight_size, center_y - highlight_size,
            self.x_pos, center_y,
            fill='white', outline=''
        )

        # Информация
        direction_text = "→ приближается" if self.direction == 1 else "← удаляется"
        info     =     f"Позиция:     {self.x_pos}px\nРазмер:     {size}px\nСкорость:
{self.speed.get():.1f}\nНаправление: {direction_text}"
        self.info_label.config(text=info)

    def animate(self):
        if self.is_moving:
            self.x_pos += self.speed.get() * self.direction

            canvas_width = self.get_canvas_width()
            min_x = 100
            max_x = canvas_width - 100

            if self.x_pos >= max_x:
                self.x_pos = max_x
                self.direction = -1
                self.status_label.config(text="● Удаляется", fg='orange')
            elif self.x_pos <= min_x:
                self.x_pos = min_x
                self.direction = 1
                self.status_label.config(text="● Приближается", fg='green')

            self.update_ball()

        self.animation_id = self.root.after(20, self.animate)

    def start_animation(self):
        self.animate()

    def toggle_motion(self):
        self.is_moving = not self.is_moving
        if self.is_moving:
            self.pause_btn.config(text="Пауза", bg='#FF9800')
            self.status_label.config(text="● Движение активно", fg='green')
        else:
            self.pause_btn.config(text="Старт", bg='#4CAF50')
            self.status_label.config(text="⏸ На паузе", fg='red')

    def reset_ball(self):
        self.x_pos = 100
        self.direction = 1
        self.is_moving = True
```

```
        self.pause_btn.config(text="Пауза", bg='#FF9800')
        self.status_label.config(text="● Приближается", fg='green')
        self.update_ball()

    def take_screenshot(self):
        try:
            if not os.path.exists("screenshots"):
                os.makedirs("screenshots")

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            filename = f"screenshots/ball_{timestamp}.png"

            # Обновляем окно чтобы получить актуальные размеры
            self.root.update()

         # Получаем координаты canvas относительно экрана
            x = self.canvas.winfo_rootx()
            y = self.canvas.winfo_rooty()
            x1 = x + self.canvas.winfo_width()
            y1 = y + self.canvas.winfo_height()

        # Делаем скриншот именно области canvas
            screenshot = ImageGrab.grab(bbox=(x, y, x1, y1))
            screenshot.save(filename)

            messagebox.showinfo("Скриншот",                                f"Скриншот
сохранён:\n{os.path.abspath(filename)}")
        except Exception as e:
            messagebox.showerror("Ошибка",       f"Не       удалось       сохранить
скриншот:\n{str(e)}")

    def on_closing(self):
        if self.animation_id:
            self.root.after_cancel(self.animation_id)
        if messagebox.askokcancel("Выход", "Закрыть окно?"):
            self.root.destroy()

    def run(self):
        self.root.mainloop()


if __name__ == "__main__":
    print("\nВыберите задание:")
    print("1 - Движущийся шар (приближение/удаление)")
    print("2 - Кривая Пеано (фрактал)")
    print("-" * 50)
    choice = input("Ваш выбор (1 или 2): ")

    if choice == "1":
        app = MovingBallApp()
        app.run()
    else:
        app = PeanoFractalApp()
        app.run()
```

```
Выберите задание:
1 - Движущийся шар (приближение/удаление)
2 - Кривая Пеано (фрактал)
--------------------------------------------------
Ваш выбор (1 или 2):
```





Вывод: были освоены возможности языка программирования Python в разработке оконных приложений.